Top K Frequent Elements & Kth Largest Element - Java Solutions

1. Top K Frequent Elements

📌 Problem Statement

Given an integer array `nums` and an integer `k`, return the k most frequent elements.

💡 Solution Approach

java

```java
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        // Count frequency of each element
        Map<Integer, Integer> map = new HashMap<>();
        for (int num : nums) {
            map.put(num, map.getOrDefault(num, 0) + 1);
        }

        // Bucket sort: index represents frequency, value contains numbers with that frequency
        List<Integer>[] bucket = new List[nums.length + 1];

        for (int i = 0; i < bucket.length; i++) {
            bucket[i] = new ArrayList<>();
        }

        // Place numbers in their corresponding frequency buckets
        for (Map.Entry<Integer, Integer> entry : map.entrySet()) {
            bucket[entry.getValue()].add(entry.getKey());
        }

        // Extract top k frequent elements from highest frequency buckets
        int[] res = new int[k];
        int idx = 0;

        for (int i = bucket.length - 1; i >= 0; i--) {
            if (bucket[i].isEmpty()) continue;

            for (int num : bucket[i]) {
                if (idx == k) {
                    return res;
                }
                res[idx++] = num;
            }
        }

        return res;
    }
}
```

🔍 Algorithm Explanation

1. **Frequency Counting**: Use a HashMap to count occurrences of each number

2. **Bucket Sort**: Create an array of lists where the index represents frequency
    - Bucket at index i contains all numbers that appear i times
3. **Result Extraction**: Iterate from highest frequency to lowest, collecting numbers until we have k elements

⏱️ Time Complexity: O(n)

- Frequency counting: O(n)
- Bucket population: O(n)
- Result extraction: O(n)

💾 Space Complexity: O(n)

- HashMap: O(n)
- Bucket array: O(n)

2. Kth Largest Element in an Array

📌 Problem Statement

Given an integer array `nums` and an integer `k`, return the kth largest element in the array.

💡 Solution Approach

java

```java
class Solution {
    public int findKthLargest(int[] nums, int k) {
        // Use min-heap to maintain top k elements
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();

        for (int num : nums) {
            minHeap.add(num);

            // Keep only k largest elements
            if (minHeap.size() > k) {
                minHeap.poll();
            }
        }

        // The root of the min-heap is the kth largest element
        return minHeap.peek();
    }
}
```

🔍 Algorithm Explanation

1. **Min-Heap Approach**: Use a min-heap (priority queue) to maintain the top k elements
2. **Heap Management**:
   - Add each element to the heap
   - If heap size exceeds k, remove the smallest element (maintaining only k largest)
3. **Result**: The root of the heap after processing all elements is the kth largest

⏱️ Time Complexity: O(n log k)

- Each insertion/removal operation: O(log k)
- Performed for n elements: O(n log k)

💾 Space Complexity: O(k)

- Heap stores at most k elements

📊 Key Differences Between the Two Problems

Aspect	Top K Frequent	Kth Largest
Goal	Find elements with highest frequency	Find single kth largest value
Data Structure	HashMap + Bucket Sort	Min-Heap
Time Complexity	O(n)	O(n log k)
Space Complexity	O(n)	O(k)
Approach	Counting + Bucketing	Heap-based selection

🎯 When to Use Which Approach

- **Top K Frequent**: When you need multiple most frequent elements and want optimal O(n) time
- **Kth Largest**: When you need a single kth largest element and can accept O(n log k) time

💡 Key Takeaways for Interview Preparation

- Bucket Sort is excellent for frequency-based problems with known range
- Min-Heap is perfect for maintaining top K elements during iteration
- Always consider time vs space tradeoffs when choosing approaches
- HashMap + Bucket Sort combination often provides O(n) solutions for frequency problems
- For Kth largest/smallest problems, heap-based solutions are generally efficient

Both solutions demonstrate efficient ways to solve "top k" problems using appropriate data structures for their specific requirements.